

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I R.Jaideep (192525441) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Jaideep

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis

Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. Hadoop Job Fails Due to Uneven Input Splits and Idle Reducers

Problem

A Hadoop job repeatedly fails because the input data is divided into uneven splits and some reducers remain idle. This indicates a problem in the way the input data or MapReduce tasks have been designed.

Likely Design Error

The main design error is **poor input partitioning and incorrect reducer configuration**. If some input splits are much larger than others, certain mapper tasks take much longer to complete. This creates a **straggler problem**, where the entire job waits for a few slow mapper tasks.

Idle reducers may indicate that:

- The number of reducers is not properly configured.
- The mapper output is highly skewed.
- Some keys occur much more frequently than others.
- The partitioning function distributes data unevenly.

For example, if one key receives most of the mapper output, a single reducer may become overloaded while other reducers remain idle.

Corrective Actions

1. Use appropriate HDFS block sizes so that input data is divided more evenly.
2. Check the input file sizes and avoid a large number of very small files.
3. Use an appropriate number of mapper and reducer tasks.
4. Use a **Combiner** to reduce intermediate data.
5. Apply a suitable partitioner to distribute keys more evenly.
6. Identify and handle data-skewed keys.
7. Monitor task execution time to identify slow or overloaded tasks.
8. Use speculative execution when appropriate to handle slow-running tasks.

Conclusion

The problem is mainly caused by **unbalanced input partitioning and improper key distribution**. Proper splitting, partitioning, combiner usage, and reducer configuration can improve parallelism and prevent reducers from remaining unnecessarily idle.

2. HDFS Data Unavailability After a Single Node Failure

Problem

An HDFS deployment experiences frequent data unavailability whenever a single DataNode fails. HDFS is designed to tolerate individual node failures, so this behavior indicates a configuration or architectural problem.

Probable Design Error

The most likely problem is an **insufficient replication factor** or incorrect replica placement.

If important HDFS blocks have only one copy, failure of the DataNode containing that copy makes the data unavailable. A replication factor of 1 provides no fault tolerance.

Another possible issue is poor replica placement. If all replicas are placed on the same physical node or within the same failure domain, a failure can affect all copies.

A NameNode design problem can also contribute if the deployment has a single NameNode without high availability.

Corrective Actions

1. Set an appropriate HDFS replication factor, commonly **3** for important data.
2. Verify that replicas are distributed across different DataNodes.
3. Enable **rack awareness** so replicas are placed across different racks.
4. Configure NameNode High Availability using Active and Standby NameNodes.
5. Monitor under-replicated and missing blocks.
6. Run HDFS rebalancing when storage becomes unevenly distributed.
7. Regularly check DataNode and NameNode health.

Conclusion

The most probable cause is **insufficient replication or incorrect replica placement**. Replication factor 3, rack-aware placement, and NameNode high availability provide better protection against individual node and infrastructure failures.

3. Duplicate Records in ETL and Apache NiFi Workflow

Problem

An ETL and Apache NiFi-based ingestion workflow creates duplicate records in the analytics store. This usually occurs when the same source data is processed more than once.

Likely Causes

1. Repeated File Processing

NiFi may read the same file multiple times if file tracking or state management is not configured correctly.

2. Pipeline Retry

When a network or database operation fails, NiFi may retry the operation. If the destination has already received the record before the failure was detected, the retry can insert the same record again.

3. Lack of Unique Identifiers

If records do not contain a unique ID, the destination system cannot easily determine whether a record has already been inserted.

4. Incorrect ETL Logic

An ETL process may repeatedly perform full loads instead of incremental loads.

5. Multiple Ingestion Paths

The same source may accidentally be connected to two different NiFi flows, causing duplicate data.

Recommended Fixes

1. Assign a **unique identifier** to every record.
2. Use NiFi state management to track successfully processed data.
3. Use appropriate NiFi processors for duplicate detection.
4. Use incremental or CDC-based database ingestion instead of repeated full loads.
5. Implement **idempotent processing**, so processing the same record multiple times produces only one final record.
6. Configure database primary keys or unique constraints.
7. Monitor failed and retried FlowFiles.
8. Ensure that the same source is not connected to multiple ingestion paths unnecessarily.

Conclusion

The duplication is generally caused by **reprocessing, retries, lack of unique identifiers, or incorrect incremental-loading logic**. State tracking, deduplication, unique keys, and idempotent ingestion can prevent duplicate records.

4. YARN Resource Starvation When Multiple Users Submit Jobs

Problem

A YARN cluster experiences resource starvation when multiple users submit jobs simultaneously. This indicates that cluster resources are not being distributed properly among users or applications.

Scheduling Error

The likely scheduling error is the use of an inappropriate scheduler configuration without proper **queues, resource limits, priorities, or fairness controls**.

If one user submits large jobs that consume most of the CPU and memory, other users may have to wait for resources. This creates resource starvation.

Better Resource Management Approach

A queue-based scheduler such as the **Capacity Scheduler** can be used to divide resources among different teams or workloads.

For example:

Queue	Resource Allocation	Priority
ETL	40%	High
Reporting	30%	Medium
ML/Research	30%	Normal

Resource limits can be applied to prevent one user or queue from consuming the entire cluster.

Corrective Actions

1. Create separate queues for different users or workloads.
2. Allocate minimum and maximum capacities to queues.
3. Set application and user resource limits.
4. Use priorities for critical jobs.
5. Prevent a single application from consuming excessive resources.
6. Allow unused resources to be temporarily shared with other queues.
7. Monitor CPU and memory utilization.
8. Use fair scheduling within queues where appropriate.

Conclusion

The problem occurs because resources are not being fairly controlled between users. A properly configured **Capacity Scheduler or Fair Scheduler**, together with queue limits and application resource limits, can prevent starvation and improve cluster utilization.

5. Backup Restores Outdated Data After Recovery

Problem

A distributed storage system restores outdated data after a failure. This means the backup or replication system is not maintaining the latest valid version of the data.

Probable Design Flaw

The most likely design flaw is the use of **stale backups or asynchronous replication without sufficient recovery-point control**.

For example, if a backup is taken once every 24 hours and the system fails before the next backup, restoring the previous backup results in loss of recent changes.

Another problem can occur if replication is delayed and the recovery process selects an older replica instead of the latest valid copy.

Corrective Actions

1. Increase Backup Frequency

Create backups at shorter intervals according to the required Recovery Point Objective (RPO).

2. Use Incremental Backups

Instead of copying the complete dataset every time, store changes since the previous backup.

3. Maintain Versioned Backups

Keep multiple versions so that the most recent valid version can be selected during recovery.

4. Monitor Replication Status

Regularly check whether replicas are synchronized and identify delayed or failed replication.

5. Use Snapshots

Storage snapshots can provide point-in-time recovery and protect against accidental data changes.

6. Define RPO and RTO

- **RPO:** Maximum acceptable amount of data loss.
- **RTO:** Maximum acceptable recovery time.

Backup frequency and replication should be designed according to these requirements.

7. Test Recovery Regularly

Backup systems should be periodically tested to verify that the restored data is current, complete, and usable.

Conclusion

The outdated recovery data is mainly caused by **infrequent backups, delayed replication, or incorrect replica selection**. Frequent incremental backups, versioning, snapshots, replication monitoring, and clearly defined RPO/RTO values can provide more reliable recovery.

Overall Conclusion

The five problems are primarily caused by incorrect Hadoop design and configuration:

- **MapReduce:** Uneven data partitioning and key distribution.
- **HDFS:** Insufficient or incorrectly placed replicas and lack of NameNode HA.
- **NiFi/ETL:** Reprocessing and missing deduplication mechanisms.
- **YARN:** Poor scheduling and resource allocation.
- **Backup:** Stale replicas or infrequent backups.

Correct configuration, monitoring, fault tolerance, balanced resource allocation, and proper recovery mechanisms are essential for building a reliable Hadoop ecosystem.